Arquitectura de Código y Directrices de Seguridad para el Desarrollo de Billeteras de Criptomonedas de Interfaz Multi-Cadena

La seguridad en el desarrollo de extensiones de navegador para criptomonedas constituye uno de los desafíos más complejos de la ingeniería de software moderna. Estos sistemas operan como agentes de custodia no de confianza que deben gestionar claves privadas y firmar transacciones financieras en entornos de ejecución intrínsecamente hostiles, como son los navegadores web de propósito general. Para garantizar la protección de activos en múltiples redes distribuidas —incluyendo Bitcoin (BTC), Ethereum (ETH), Solana (SOL) y BNB Chain (BNB)—, es indispensable adoptar una arquitectura de código defensiva, alineada con los estándares criptográficos globales y blindada contra vectores de explotación físicos y lógicos.

Arquitectura de Referencia y Fuentes de Código de Billeteras Web3

Las implementaciones de referencia de código abierto en el ecosistema Web3 demuestran que las extensiones más maduras dividen sus sistemas en componentes desacoplados para aislar el estado privilegiado. Esta compartimentación estructural evita que un compromiso en la interfaz gráfica o una inyección de secuencias de comandos en sitios cruzados (XSS) resulte en la extracción directa de las llaves del usuario.

Billetera de Ref	Origen del Código y Repositorios Clave	Soporte Nativo de	Arquitectura de Mensajería y Dependencias Críticas

ere nci a		Red es	

Met aMa sk	`MetaMask/metamask-extension`
	
	`MetaMask/KeyringController`
	
	`MetaMask/eth-phishing-detect`
		Ethereum (EVM), BNB Chain, Polygon, Arbitrum, Optimism	Arquitectura basada en controladores (`KeyringController`). Utiliza Yarn con Corepack y Node.js para compilar entornos distribuidos en Chromium y Firefox. Protegido a nivel de dependencias mediante la herramienta LavaMoat.

Rab by Wall et	`RabbyHub/Rabby`
	
	`RabbyHub/rabbykit`
		Ethereum y más de 141 cadenas compatibles con EVM	Estructura modular dividida en `walletController` y `providerController`. Emplea un script de contenido (`content-script`) que actúa como puente de mensajes de baja latencia entre la página web y el Service Worker central.

Backpack	`coral-xyz/backpack`	Solana (SVM), Ethereum (EVM), Bitcoin (BTC)	Monorepo de TypeScript diseñado para soportar extensiones ejecutables (xNFTs). Implementa un entorno de recarga en vivo con certificados locales auto-firmados (`mkcert`) para el desarrollo seguro fuera de línea.
UniSat	`unisat-wallet/wallet`	Bitcoin (BTC, Ordinals, BRC-20)	Billetera especializada en la gestión de UTXOs y firma de transacciones complejas de Bitcoin sobre código TypeScript y Less.

El gran reto arquitectónico al que se enfrentan las extensiones de billetera multi-cadena radica en las diferencias fundamentales de diseño entre las redes blockchain. Las cadenas basadas en la Ethereum Virtual Machine (EVM), como Ethereum y BNB Chain, comparten un modelo de cuenta uniforme, el mismo formato de dirección y APIs de ejecución idénticas, lo que simplifica la integración mediante librerías como `ethers.js`, `web3.js` o `viem`. En contraste, redes no-EVM como Solana emplean el entorno de ejecución SVM y esquemas de firma basados en Ed25519, mientras que Bitcoin se fundamenta en un modelo de Salidas de Transacciones No Gastadas (UTXO) que requiere la reconstrucción y firma manual de entradas mediante transacciones de Bitcoin parcialmente firmadas (PSBT).

Estándares Criptográficos de Derivación e Interoperabilidad

Para garantizar que un usuario pueda restaurar y gestionar sus activos en múltiples redes utilizando un único secreto de respaldo, el código de las billeteras debe estructurarse conforme a propuestas de mejora estandarizadas de Bitcoin (BIP) y Ethereum (EIP).

Generación de Entropía y Semilla Nemotécnica (BIP-39)

La generación de la frase de recuperación semilla comienza con la recolección de entropía físicamente segura a nivel del sistema operativo. El estándar prohíbe el uso de funciones pseudoaleatorias comunes de los lenguajes de programación (como `Math.random` en JavaScript), exigiendo en su lugar APIs criptográficas nativas del sistema. Una cantidad de entropía de 128 a 256 bits se somete a un cálculo de suma de comprobación (checksum) derivando su hash SHA-256 :

$$\text{Checksum} = \text{SHA-256}(\text{Entropy})$$

La entropía concatenada con el checksum se divide en segmentos de 11 bits, donde cada segmento se traduce en un índice de número entero entre 0 y 2047 que apunta a una palabra específica dentro de un diccionario estandarizado de 2048 términos.

Posteriormente, la frase de recuperación se convierte en una semilla binaria de 512 bits aplicando la función de derivación de claves basada en contraseñas 2 (PBKDF2) utilizando HMAC-SHA512 :

$$\text{Seed} = \text{PBKDF2-HMAC-SHA512}(\text{Password}=\text{Mnemonic}, \text{Salt}=\text{"}$$

$$\text{mnemonic"+Passphrase,Iterations=2048)}$$

El uso de una frase de contraseña opcional (comúnmente denominada la palabra número 25) altera radicalmente la semilla resultante, sirviendo como una medida de seguridad efectiva para la denegación plausible en caso de coacción física.

Derivación Jerárquica Determinista (BIP-32, BIP-43 y BIP-44)

Una vez obtenida la semilla maestra de 512 bits, el estándar BIP-32 define la creación de un árbol infinito de claves privadas y públicas. La semilla se procesa inicialmente mediante HMAC-SHA512 utilizando la constante binaria `"Bitcoin seed"` para generar la clave privada maestra (m) y el código de cadena maestro. El código de cadena (chain code) de 256 bits introduce entropía adicional en cada nivel del árbol de derivación.

La ruta de acceso a las claves para dar soporte multi-cadena de forma estructurada se rige por el estándar BIP-44 a través de la siguiente jerarquía fija de cinco niveles :

$$m/\text{purpose}$$

$$'$$

$$/\text{coin_type}$$

$$'$$

$$/\text{account}$$

$$'$$

/change/address_index

Donde el apóstrofe (
′
) denota una derivación endurecida (hardened derivation). En las claves endurecidas, el índice del nodo hijo es mayor o igual a 2^{31}. La derivación normal permite calcular claves públicas hijas a partir de una clave pública padre y el código de cadena, facilitando la auditoría de saldos. Sin embargo, si una sola clave privada hija normal se ve comprometida junto con el código de cadena padre, un atacante matemático puede deducir la clave privada padre. El uso estricto de derivación endurecida en los primeros tres niveles del árbol impide esta fuga colateral.

Red Blockchain	Tipo de Moneda (`coin_type`)	Estándar de Derivación	Formato de Dirección Resultante
Bitcoin (BTC)	`0'`	BIP-84 (Native SegWit)	Direcciones Bech32 (`bc1q...`) basadas en testigos.
		BIP-86 (Taproot)	Direcciones Taproot (`bc1p...`) con firmas Schnorr.
Ethereum (ETH)	`60'`	BIP-44	Cuentas basadas en el formato hexadecimal de Ethereum (`0x...`).
Solana (SOL)	`501'`	BIP-44 (Curva Ed25519)	Direcciones codificadas en Base58 representativas de puntos sobre la curva Ed25519.

BNB Chain (BNB)	`714'`	BIP-44 (Compatibilidad EVM)	Direcciones con formato idéntico a las de Ethereum (`0x...`) compartiendo el mismo modelo de cuentas.

Implementación Práctica del Ciclo de Mensajería Seguro (Manifest V3)

La transición obligatoria a la especificación de extensiones Manifest V3 en navegadores modernos impone restricciones estrictas sobre la persistencia de los scripts de fondo, eliminando las páginas de fondo persistentes en favor de Service Workers de ciclo de vida transitorio. Esto exige una reestructuración completa de los flujos de comunicación.

Configuración del Manifiesto de la Extensión (`manifest.json`)

El archivo de manifiesto debe declarar de manera explícita y restrictiva los permisos necesarios, impidiendo el uso de comodines innecesarios que expandan la superficie de ataque ante inyecciones de código :

JSON

```
  "manifest_version"  3
  "name"  "Billetera Multi-Cadena Segura"
  "version"  "1.0.0"
  "background"
    "service_worker"  "background/service_worker.js"
    "type"  "module"

  "permissions"
  "content_scripts"

      "matches"   "file://*"  "http://*/*"
"https://*/*"
      "js"   "content/content_script.js"
      "run_at"  "document_start"
      "all_frames"  true


  "web_accessible_resources"

      "resources"   "inpage/provider.js"
      "matches"   "<all_urls>"
```

Script de Fondo (`background/service_worker.js`) y

Mensajería Segura

El Service Worker debe configurarse para responder de forma asíncrona a los mensajes enviados por el script de contenido. Es mandatorio verificar minuciosamente el origen del remitente para evitar que una página maliciosa simule ser un componente interno de la extensión.

JavaScript

```
// background/service_worker.js
import                          from
'./controllers/keyring.js'

const                       new

// Escucha de mensajes del script de contenido o popups
internos


  // Verificación crítica de seguridad: validar que el
remitente proviene de la propia extensión
  const

  if
    // Si la llamada no es interna, validar el origen web
a través de la pestaña emisora
    const
                 null
    if
                      error  'Origen no verificable'
      return false
```

```
  // Patrón de ejecución asíncrono en Manifest V3

                                        data
                                        error


  // Retornar true mantiene el canal de mensajería
abierto para la respuesta asíncrona
  return true


async function handleAction

  switch
    case 'eth_requestAccounts'
      // Validar si el origen no está bloqueado y
despertar la ventana emergente de confirmación
      return await

    case 'personal_sign'
      if
        // Las peticiones externas deben pasar
obligatoriamente por la UI de autorización del usuario
        return await
```

```
      return await

    default
      throw new Error 'Método no soportado'
```

Mitigación de Vectores de Ataque Criptográficos y de Memoria

El desarrollo de software financiero sobre entornos web expone a las billeteras a ataques altamente sofisticados orientados a la exfiltración de claves mediante el volcado físico del almacenamiento o fallas lógicas en las firmas.

1. Ataques de Reutilización de Nonce (Phantom Signature Attacks)

En los algoritmos de firma basados en curvas elípticas, como ECDSA para la curva secp256k1, el emisor de la transacción debe generar un parámetro aleatorio secreto denominado nonce (k) para cada firma individual. Las ecuaciones criptográficas de firma para dos transacciones independientes se formulan de la siguiente manera, donde r representa la coordenada en el eje de las abscisas del punto de la curva elíptica derivado de k, d es la clave privada del usuario, e
1
,e
2
son los hashes de los mensajes de las transacciones y s
1

$,s$

2

corresponden a la firma resultante :

s

1

$\equiv k$

-1

$\cdot (e$

1

$+r \cdot d)(\mathrm{mod} n)$

s

2

$\equiv k$

-1

$\cdot (e$

$$2$$

$$+r \cdot d)(\mathrm{mod} n)$$

Si el generador pseudoaleatorio de la billetera es deficiente y reutiliza el mismo nonce k para firmar dos transacciones con diferentes cargas útiles ($e_1 = e_2$), el valor de r permanece idéntico en ambas firmas. Un atacante externo que monitorice la red blockchain puede capturar estas firmas y calcular algebraicamente el nonce efímero k sin conocer la clave privada del usuario :

$$s$$

$$1$$

$$-s$$

$$2$$

$$\equiv k$$

$$-1$$

$$\cdot (e$$

1

−*e*

2

)(mod*n*)

k≡(*s*

1

−*s*

2

)

−1

·(*e*

1

−*e*

2

$$)(\mathrm{mod}n)$$

Una vez extraído el valor de k, la deducción de la clave privada maestra del usuario (d) se completa de forma matemática instantánea :

$$d \equiv r$$

-1

$\cdot (s$

1

$\cdot k - e$

1

$$)(\mathrm{mod}n)$$

Este vector de ataque, clasificado como CWE-347, se neutraliza de forma absoluta exigiendo la generación determinista de firmas bajo la especificación del estándar **RFC-6979**. Las firmas deterministas computan el valor de k combinando la clave privada junto con el resumen hash del mensaje a través de un proceso iterativo gobernado por HMAC-SHA256, eliminando por completo la necesidad de un generador de aleatoriedad en tiempo de firma.

Adicionalmente, se debe considerar que los análisis forenses de librerías criptográficas de JavaScript revelan que `noble-curves` realiza la reducción modular del hash del mensaje antes de la generación del nonce k, mientras que repositorios históricos como `RustCrypto` y `eth-keys` la ejecutan después. Esta discrepancia técnica produce firmas divergentes para un mismo mensaje cuando el valor de su hash supera el orden de la curva elíptica, lo cual exige que los ingenieros sincronicen los flujos de reducción modular al realizar pruebas de compatibilidad cruzada entre entornos multi-cadena.

2. La Vulnerabilidad Demónica (CVE-2022-32969) y Fuga en Disco

Esta vulnerabilidad surge debido al comportamiento de los navegadores web modernos al gestionar el almacenamiento temporal ante interrupciones de servicio y restauración de pestañas. Los motores de renderizado de Chromium y Firefox guardan periódicamente en el disco local el contenido en texto plano de todos los elementos `<textarea>` y campos de entrada de datos que no estén configurados específicamente como tipos de contraseña protegidos.
Si un usuario importa o visualiza su frase de recuperación semilla de 12 o 24 palabras en un elemento de texto abierto convencional, la frase se escribe de manera persistente en disco en texto plano, accesible para atacantes locales o troyanos capaces de leer la carpeta de configuración local del navegador, persistiendo incluso tras desinstalar la extensión o reiniciar el equipo de cómputo.

Mitigación Técnica en Interfaz de Usuario (UI):

Entradas de Texto Segmentadas: Prohibir la captura de frases de recuperación en elementos de texto únicos. Se deben implementar campos de tipo `<input type="password">` independientes para cada una de las palabras, de modo que el navegador interprete el campo como un secreto de alta seguridad y restrinja su volcado automático al sistema de archivos.
Neutralización de Portapapeles: Bloquear la acción de copiado y

pegado directo de la frase semilla completa en la interfaz mediante el desvío controlado de eventos del DOM :

JavaScript

```
const
document                    'mnemonic-word-1'

                                'paste'

  // Impedir la persistencia del portapapeles en la
memoria del navegador


        'Por razones de seguridad, introduzca la palabra
utilizando el teclado.'
```

Evitar Carga Directa en el DOM: Cuando se muestre la frase semilla por pantalla al usuario durante la fase de creación, evitar pintarla en forma de texto HTML estándar. En su lugar, se puede dibujar cada palabra de forma gráfica en un lienzo bidimensional (`<canvas>`), forzando a que la representación en el árbol de nodos de la memoria del navegador contenga únicamente mapas de bits sin representación semántica legible por scripts inyectados o volcadores de memoria del renderizador.

Gestión de Memoria y Almacenamiento Seguro de Claves

Un principio fundamental de la seguridad en el desarrollo web prohíbe el uso de `localStorage` o `sessionStorage` para almacenar claves criptográficas o datos de sesión críticos. Al ser APIs sincrónicas accesibles de forma global por cualquier secuencia de comandos dentro del mismo origen, cualquier vulnerabilidad de inyección de código (XSS) o ataque a la cadena de suministro permite la lectura inmediata y extracción remota de los secretos allí alojados.

Criptografía Web no Extraíble con IndexedDB

Para garantizar la máxima protección en el cliente, las billeteras deben almacenar el almacén de claves (vault) cifrado dentro de `IndexedDB` utilizando llaves de cifrado generadas bajo la API nativa de criptografía web (`Web Crypto API`) marcadas estrictamente como no exportables.

JavaScript

```
// db/secure_store.js

// Generar una clave de cifrado de contenido (CEK) simétrica de tipo AES-GCM
// Al definir extractable como false, el navegador impide físicamente que el
// motor de JavaScript acceda o exponga la clave privada en texto plano.
async function generateNonExtractableKey
  const       await window

      name  'AES-GCM'
      length  256

    false  // extractable: false (La clave NUNCA podrá ser leída por JS land)
     'encrypt'  'decrypt'

  return
```

```
// Cifrar los datos sensibles del llavero
async function encryptVault
  const           new
  const
              JSON

  // Generación de un vector de inicialización (IV)
criptográficamente seguro
  const      window                          new
Uint8Array 12

  const              await window

      name   'AES-GCM'
      iv




  return
    iv  Array
    ciphertext  Array      new Uint8Array
```

Para dotar al sistema de protección por hardware de nivel militar en dispositivos

móviles u ordenadores portátiles de última generación, las billeteras nativas pueden integrarse con el enclave seguro (Secure Enclave). El Secure Enclave es un chip de hardware aislado del procesador principal que genera y opera pares de claves elípticas NIST P-256 de forma autónoma. Al configurar la clave bajo el parámetro `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`, la clave privada jamás abandona el silicio físico del enclave. La extensión únicamente envía los datos de la transacción al enclave y recibe de vuelta el resultado de la firma criptográfica, mitigando por completo las vulnerabilidades de lectura de memoria RAM.

Análisis Forense de Memoria y Condiciones de Carrera

Auditorías detalladas dirigidas por Least Authority al código de MetaMask descubrieron debilidades extremas relacionadas con la inestabilidad del sistema y la concurrencia :

1. **Condiciones de Carrera de LevelDB:** Al desinstalar la extensión de MetaMask en entornos Windows, el navegador Chromium inicia la eliminación de la carpeta local de configuraciones en un hilo del sistema operativo independiente. Si la base de datos LevelDB intenta realizar una escritura en caliente del diario de transacciones concurrentemente antes de que el comando de remoción se complete, la API de archivos de Windows aborta de forma silenciosa la eliminación del directorio al estar los archivos en uso. En la siguiente instalación de la billetera, el diario no eliminado es renombrado como `LOG.old`, permitiendo a un atacante con acceso al equipo leer el archivo residual y recuperar la semilla cifrada del usuario anterior.
2. **Acoplamiento de Métodos de Inicialización:** Se detectó que el método asíncrono `addAccounts` de la clase `HD Keyring` gestionaba simultáneamente la derivación de cuentas nuevas y la verificación de la inicialización de la semilla. Ante un fallo de red o error de memoria del navegador durante el desbloqueo, este acoplamiento forzaba una inicialización por defecto que generaba de forma silenciosa una nueva semilla nemotécnica aleatoria, sobrescribiendo de manera irreversible la semilla legítima del usuario almacenada en memoria activa y provocando

la pérdida total de los fondos asociados al cerrarse la pestaña.

Triple Protección del Tiempo de Ejecución (LavaMoat y Cadena de Suministro)

La sofisticación de los ataques dirigidos a plataformas Web3 ha desplazado su foco principal hacia el compromiso de las dependencias externas integradas en la fase de compilación. Para impedir la ejecución incontrolada de dependencias vulnerables o comprometidas que busquen interceptar la clave criptográfica en caliente, se exige un modelo de triple protección en el desarrollo.

Implementación del Sandbox del Tiempo de Ejecución Mediante LavaMoat

LavaMoat actúa como un cortafuegos interno sobre el empaquetador del proyecto mediante la aplicación del modelo de seguridad basado en capacidades de objeto (Object Capability Security). El sistema se configura de manera que los scripts de las librerías de terceros instaladas se ejecutan dentro de compartimentos de aislamiento lógico creados mediante JavaScript Endurecido (Hardened JavaScript - SES).

Para proteger el flujo de instalación contra dependencias con comportamientos maliciosos, los desarrolladores deben añadir restricciones específicas dentro del manifiesto de dependencias `package.json`:

JSON

```
  "name"  "billetera-segura"
  "version"  "1.0.0"
  "scripts"
    "lavamoat-policy"  "lavamoat-node generate-policy"
    "build:secure"  "browserify index.js -p
lavamoat-browserify > bundle.js"

  "devDependencies"
    "@lavamoat/allow-scripts"  "^2.0.0"
    "lavamoat-browserify"  "^15.0.0"
```

```
"lavamoat"
  "allow-scripts"
    "dependencies"
      "keccak"  true
      "secp256k1"  true
```

La protección en tiempo de ejecución gobernada por LavaMoat y SES mitiga activamente ataques catastróficos a la cadena de suministro mediante tres controles críticos :

- **Bloqueo de Modificación de Primordiales (Prototypes):** Impide que librerías maliciosas alteren objetos globales nativos del motor de JavaScript (como `Object.prototype` o `Array.prototype`), evitando que intercepten buffers de memoria que contengan claves privadas.
- **Atenuación de Acceso a APIs del Navegador:** Cada paquete importado solo tiene acceso a las APIs declaradas explícitamente en su archivo de política. Si una librería criptográfica comprometida intenta, tras una actualización maliciosa, acceder al objeto `window.location` o realizar una petición `fetch` externa para exfiltrar secretos a un servidor de comando y control (C2), LavaMoat deniega la llamada de forma automática al no estar permitida por su política explícita.
- **Scuttling (Scuttling Security Feature):** LavaMoat ofrece un modo de seguridad avanzado que elimina o inutiliza sistemáticamente el acceso a primordiales expuestos no utilizados dentro del entorno global tras la fase de inicialización de la extensión, reduciendo al mínimo la superficie de ataque explotable por inyecciones en tiempo de ejecución.

Este enfoque de defensa en profundidad se complementa preventivamente

mediante integraciones con el analizador basado en inteligencia artificial de `Socket.dev`, capaz de alertar en tiempo de integración y de forma proactiva sobre anomalías de red o código malicioso insertado en paquetes recién publicados antes de que alcancen los entornos de producción.

Conclusiones y Recomendaciones de Ingeniería de Seguridad

La robustez de una extensión de billetera multi-cadena depende enteramente de la eliminación de supuestos optimistas respecto al entorno donde opera. Los desarrolladores deben asumir que el dispositivo y el navegador del cliente se encuentran lógicamente comprometidos.

Para lograr un sistema de alta seguridad, es indispensable estructurar el código bajo un enfoque de privilegios mínimos utilizando las APIs nativas no extraíbles de Criptografía Web, forzando firmas deterministas gobernadas por el estándar RFC-6979 para neutralizar ataques matemáticos de recuperación de claves. Las interfaces de usuario de confirmación deben diseñarse con controles visuales estrictos de origen e implementarse bajo arquitecturas de empaquetado seguro como LavaMoat para garantizar el aislamiento físico de las dependencias de terceros. La observancia estricta de estas directrices criptográficas y arquitectónicas constituye la única defensa viable para salvaguardar los activos digitales en redes distribuidas.

Esta investigación exhaustiva analiza la arquitectura, los estándares de desarrollo, los vectores de ataque y las prácticas de implementación segura para carteras de criptomonedas (wallets) en forma de extensiones de navegador, cubriendo ecosistemas clave como **Bitcoin (BTC)**, **Ethereum (ETH)**, **Solana (SOL)** y **BNB Chain (BNB)**.

1. Deconstrucción de la Arquitectura de una Extensión Wallet

Las extensiones de cartera no son aplicaciones monolíticas; operan divididas en contextos de ejecución completamente aislados por el navegador web. Garantizar la seguridad implica entender cómo interactúan estos componentes:

- **In-Page Provider (Contexto de la DApp):** Un script que se inyecta directamente en el DOM de la página web del usuario. Expone una API estándar (como `window.ethereum` o `window.solana`) para que los sitios web detecten la wallet, soliciten cuentas y propongan transacciones.
- **Content Script (El Puente):** Actúa como intermediario. Se ejecuta en un "mundo aislado" (Isolated World); comparte el acceso al DOM con la página web, pero no las variables de JavaScript ni el estado del runtime. Su única función debe ser canalizar mensajes RPC cifrados/validados entre la DApp y el núcleo de la wallet.
- **Background Script / Service Worker (El Núcleo Seguro):** Es el único componente persistente y con acceso privilegiado a las APIs de almacenamiento seguro de la extensión (`chrome.storage.local`). **Aquí reside la lógica criptográfica, el descifrado del almacén de claves (vault) y la generación de firmas.** Nunca expone claves privadas a los Content Scripts ni a la interfaz de usuario de forma directa.
- **Popup / UI (Interfaz de Usuario):** La ventana emergente que se abre cuando el usuario hace clic en el icono o cuando se requiere una aprobación de transacción. Se comunica directamente con el Background Script mediante canales IPC internos de la extensión.

2. Repositorios de Código Abierto de Referencia

Para analizar código de producción robusto, los siguientes repositorios abiertos representan el estado del arte en la ingeniería de wallets:

Ecosistema EVM (Ethereum & BNB Chain)

- **MetaMask Extension:** El estándar de la industria para redes EVM. Su arquitectura de controladores modulares separa la gestión de claves del estado de la red.
  Enlace: `https://github.com/MetaMask/metamask-extension`

Rabby Wallet: Destaca por su enfoque de seguridad proactiva, simulación de transacciones antes de firmar y separación estricta mediante controladores dedicados (`walletController` y `providerController`).

Enlace: `https://github.com/rabbyHub/rabby`

Ecosistema Multicadena y Solana

Backpack: Desarrollada principalmente en TypeScript, es una de las arquitecturas multicadena (Solana, EVM, BTC) más modernas, construida bajo el concepto de xNFTs y con una gestión modularizada de proveedores de red.

Enlace: `https://github.com/coral-xyz/backpack`

Ecosistema Bitcoin

UniSat Wallet: Cartera de código abierto optimizada para Bitcoin, compatible con transacciones nativas de BTC, Ordinals y tokens BRC-20.

Enlace: `https://github.com/unisat-wallet/wallet`

3. Estándares de Desarrollo y SDKs Vitales

El núcleo matemático y operativo de cualquier cartera criptográfica se rige por propuestas de mejora de Bitcoin (**BIPs**) y de Ethereum (**EIPs**):

Criptografía y Derivación de Claves (BTC, ETH, BNB, SOL)

BIP-39 (Mnemonic Code): Define la generación de una secuencia de entropía aleatoria (128 a 256 bits) codificada en una frase mnemónica legible por humanos (12 a 24 palabras) extraída de un diccionario de 2,048 palabras indexadas. Esta frase se transforma en una semilla de 512 bits mediante PBKDF2 con HMAC-SHA512.

BIP-32 / BIP-44 (Hierarchical Deterministic Wallets): Permiten derivar un árbol infinito de claves públicas y privadas a partir de la semilla única de 512 bits utilizando una ruta estructurada: `m / purpose' / coin_type' / account' / change / address_index`

Rutas de derivación estándar por cadena:

Cadena	Moneda / Tipo	Ruta Estándar	Curva Elíptica
BTC	Bitcoin (Legacy)	`m/44'/0'/0'/0/0`	secp256k1
BTC	Bitcoin (Native SegWit)	`m/84'/0'/0'/0/0`	secp256k1
BTC	Bitcoin (Taproot)	`m/86'/0'/0'/0/0`	secp256k1 (Schnorr)
ETH	Ethereum	`m/44'/60'/0'/0/0`	secp256k1
BNB	BNB Chain (EVM)	`m/44'/60'/0'/0/0`	secp256k1
SOL	Solana	`m/44'/501'/0'/0'`	ed25519

BIP-174 (Partially Signed Bitcoin Transactions - PSBT): Formato binario estándar para pasar transacciones de Bitcoin entre diferentes entidades (como una dApp, una extensión wallet y un hardware wallet) para que añadan firmas e inputs de forma independiente antes de la extracción de la transacción final de la red.

Inyección de Proveedores y Descubrimiento en el Navegador

EIP-1193 (Ethereum Provider JavaScript API): Define una interfaz común basada en RPC para exponer servicios Ethereum a las DApps. Utiliza el método `.request({ method, params })`.

EIP-6963 (Multi-Injected Provider Discovery): Resuelve las colisiones críticas que ocurrían cuando múltiples wallets instaladas (ej. MetaMask y Rabby) intentaban sobrescribir simultáneamente el objeto global `window.ethereum`. Introduce un sistema de descubrimiento bidireccional basado en eventos estandarizados nativos de JavaScript (`eip6963:announceProvider` y `eip6963:requestProvider`).

4. Vectores de Ataque Críticos y Vulnerabilidades Reales

A. Ataques a la Cadena de Suministro de Software (Supply Chain Attacks)

Las wallets basadas en JavaScript dependen de cientos de librerías de terceros a través de `npm`. Si un atacante compromete una dependencia menor (mediante robo de credenciales del mantenedor o typosquatting), puede inyectar código malicioso oculto que detecte cuándo se descifra la clave privada en memoria para exfiltrarla a un servidor C2 externo.

Caso real: El ataque al Ledger Connect Kit, donde una versión comprometida inyectó un drainer malicioso que alteraba las direcciones de destino en los payloads de transacciones legítimas.

B. La Vulnerabilidad "Demonic" (CVE-2022-32969)

Descubierta en múltiples carteras basadas en extensiones de navegador.

Causa raíz: Cuando un usuario ingresaba su frase mnemónica en un elemento estándar `<textarea>` o campos de texto desprotegidos durante el proceso de restauración, algunos motores de navegación capturaban el búfer del DOM y escribían el texto plano en los archivos de almacenamiento del sistema ("Restore Session Cache") en el disco duro. Si el equipo sufría un compromiso lógico posterior, un malware local podía

extraer las palabras semilla en texto plano directamente del disco duro, evadiendo el cifrado AES de la wallet.

C. Vulnerabilidades de Mensajería Inter-Proceso (Extension IPC Hijacking)

Si los escuchas de mensajes internos instalados en el Background Service Worker (`chrome.runtime.onMessage.addListener` o `chrome.runtime.onConnectExternal`) no realizan una validación rigurosa del origen del remitente, un sitio web malicioso o una extensión colindante comprometida puede forzar llamadas RPC internas simulando comandos de transacciones o solicitudes de firma de mensajes de identidad.

D. Ataque criptográfico por reutilización de Nonce (k-collision)

En firmas basadas en ECDSA (secp256k1) utilizadas por BTC y ETH, cada firma requiere generar un número aleatorio único llamado **nonce criptográfico (k)**. Si un software de wallet implementa un generador pseudoaleatorio débil y reutiliza el mismo valor de k para firmar dos transacciones diferentes con la misma clave privada, un atacante puede deducir algebraicamente la clave privada exacta analizando las dos firmas públicas en la blockchain.

La ecuación matemática involucrada en la generación del parámetro de firma s es:

$$s = k$$

$$-1$$

$$(e + r \cdot d) (\mathrm{mod} n)$$

Donde e es el hash del mensaje y d es la clave privada. Si el valor de k se duplica en otra transacción con hash e

′

, los valores de s y s

′

permiten aislar analíticamente y calcular con precisión milimétrica la clave privada del usuario (d).

5. Análisis de Auditorías de Seguridad de Código Abierto

Estudiar los informes de auditorías emitidos por firmas como Least Authority, Trail of Bits o Halborn proporciona patrones de vulnerabilidades estructurales recurrentes:

Auditoría de la Implementación de Semillas en MetaMask (Least Authority)

- **Fallo estructural detectado:** Se identificó la existencia de condiciones de carrera (race conditions) en el motor de persistencia LevelDB local en sistemas Windows. Cuando un usuario desinstalaba la extensión, ciertos archivos de registros dispersos (log segments) mantenían fragmentos del almacén cifrado en búferes huérfanos que el sistema operativo bloqueaba y no borraba de inmediato, dejando vectores expuestos para análisis forense de memoria en frío.
- **"Seed Phrase Errors":** Acoplamientos lógicos erróneos dentro de funciones internas como `addAccounts` y `_initFromMnemonic` provocaban fugas menores donde, bajo excepciones específicas de memoria, direcciones de sub-cuentas corruptas se entrelazaban debido a fallos en la alineación de índices de derivación.

Auditoría de MetaMask Snaps (Least Authority)

- **Enfoque de aislamiento:** Evalúa el sistema de sandboxing para código ejecutable de terceros (Snaps). Se recomendó el endurecimiento riguroso del entorno **SES (Secure EcmaScript)** para mitigar vulnerabilidades donde las dependencias importadas intentaban redefinir prototipos globales integrados de JavaScript (`Object.prototype`) para romper el

aislamiento del aislamiento de memoria (prototype pollution).

6. Guía de Implementación Segura y Mitigación de Código

A continuación se exponen las contramedidas de desarrollo con código TypeScript conceptual para blindar el funcionamiento del núcleo de una wallet:

Generación de Entropía Criptográficamente Segura (CSPRNG)

Nunca debe utilizarse `Math.random()`. Se debe invocar la API criptográfica del sistema operativo a través del entorno web del navegador.

TypeScript

```
import                                          from
'bip39'


/**
 * Genera una frase semilla BIP-39 de forma segura
garantizando CSPRNG robusto
 */
export function createSecureWallet                    string

  // Generar 256 bits de entropía puramente aleatoria del
sistema
  const           new Uint8Array 32



  // Convertir la entropía segura en una frase de 24
palabras mnemónicas
  const                                 256
    // Retornar un buffer con la entropía generada por
```

```
CSPRNG
    return                                  0


  // Generar la semilla raíz de 512 bits
  const

  return
```

Mitigación de Vulnerabilidad "Demonic": Campos de Entrada Protegidos

Para evitar la caché interna de entrada de texto del navegador en disco, se deben fragmentar las palabras semilla en inputs individuales con enmascaramiento estricto:

HTML

```
    div class "mnemonic-word-container"
      input
       type "password"
       autocomplete "off"
       autocorrect "off"
       autocapitalize "off"
       spellcheck "false"
       name "bip39-word-1"
       id "word-1"
       class "secure-input-field"
```

```
    div
```

Además, se debe programar un interceptor en el evento `onPaste` dentro de la UI para capturar la entrada, inyectarla directamente a estructuras de datos internas en memoria volátil de JavaScript (ej. un array de strings en el estado de React) y limpiar el portapapeles del sistema operativo de inmediato mediante la API del Clipboard:

TypeScript

```
  const                async

                       // Detiene la propagación y
previene que el búfer toque el DOM
    const                                    'text'
''
    const                                /\s+/


    if                   12                  24
      // Guardar directamente en variables internas en
memoria volátil


      // Borrar de forma inmediata el portapapeles para
evitar malware husmeador
      await                              ''
```

Almacenamiento Seguro en Reposo: Claves No Exportables (Web

Crypto API)

No guardes contraseñas ni frases semilla descifradas en `chrome.storage.local` ni en `localStorage`. El secreto maestro debe estar cifrado en reposo utilizando **AES-GCM de 256 bits**. La clave de cifrado simétrico derivada de la contraseña del usuario nunca debe persistir en texto plano; en su lugar, se genera mediante la Web Crypto API especificando que la clave resultante sea estrictamente no extraíble (`extractable: false`).

TypeScript

```
/**
 * Cifra el Vault que contiene los datos sensibles de la wallet
 */
export async function encryptVault           string
                  string   Promise
ArrayBuffer      Uint8Array        Uint8Array

  const       new

  const                                 new
Uint8Array 16

  const                                 new Uint8Array 12
// IV de 12 bytes recomendado para AES-GCM


  // Importar el texto plano de la contraseña como base de clave raw
  const           await
    "raw"


      name   "PBKDF2"

    false
```

```
    "deriveKey"


  // Derivar clave de cifrado simétrico robusto AES-GCM de 256 bits
  const             await

      name  "PBKDF2"
      salt
      iterations  600000  // Alto número de iteraciones de hashing para mitigar ataques de fuerza bruta
      hash  "SHA-256"


      name  "AES-GCM"  length  256
    false  // CRÍTICO: extractable: false impide que la clave de cifrado sea leída por scripts maliciosos o volcados
    "encrypt"  "decrypt"


  const              await
      name  "AES-GCM"  iv



  return
```

Firmas Deterministas Inmunes a Colisión de Nonce: RFC 6979

Para evitar por completo la exposición de claves privadas por reutilización de nonces defectuosos en curvas `secp256k1`, la wallet debe prescindir de fuentes de aleatoriedad externas al firmar y usar el estándar **RFC 6979**. Este algoritmo computa el nonce k de manera determinista combinando mediante operaciones HMAC-SHA256 la propia clave privada y el hash del mensaje a firmar.

Librerías modernas auditadas como `@noble/curves` de Paul Miller (utilizadas extensamente en las dependencias actuales del ecosistema Web3/EVM) garantizan el cumplimiento automático de firmas deterministas bajo RFC 6979.

TypeScript

```
import                  from '@noble/curves/secp256k1'


/**
 * Firma un hash de transacción utilizando firmas
deterministas secp256k1 (RFC 6979)
 */
export function signTransactionSecure
Uint8Array              Uint8Array   Uint8Array
  // @noble/curves implementa internamente RFC 6979 para
mitigar los ataques de reutilización de nonce k
  const


  return                      // Retorna la firma
codificada en formato DER estándar para blockchain
```

7. Arquitectura de Extensión Inmune (Manifest V3 + LavaMoat)

Para blindar la infraestructura contra inyecciones e intrusiones externas, se deben aplicar de forma rigurosa las capacidades de confinamiento modernas proporcionadas por el motor del navegador:

A. Validación Estricta de Remitentes en Mensajería IPC

El Background Service Worker debe actuar como una aduana. Cada mensaje entrante debe verificar la identidad del emisor:

TypeScript

```
  // 1. Mitigación de Extension Hijacking: Verificar que el mensaje provenga de nuestra propia UI/Popup
  if
    console       "Intento de violación de origen detectado por extensión externa."
    return false


  // 2. Procesar solicitudes internas legítimas (ej: abrir popup de confirmación)
  if                      "REQUEST_SIGNATURE"
```

B. Implementación Obligatoria de Manifest V3 (MV3)

Manifest V3 elimina la capacidad heredada de ejecutar código arbitrario remoto, reduciendo drásticamente la superficie de ataque:

- Prohíbe por completo el uso de `eval()` o `new Function()` mediante directivas estrictas de la **Política de Seguridad de Contenido (CSP)** por defecto del navegador.
- Prohíbe la inyección de código desde servidores remotos (`unsafe-remote-code`). Todo el árbol lógico de ejecución de la wallet debe estar compilado localmente dentro del paquete ejecutable empaquetado `.crx` verificado por la tienda del navegador.

C. Contención de Dependencias en Tiempo de Ejecución: LavaMoat

Para resolver de raíz los ataques a la cadena de suministro en el ecosistema JavaScript, MetaMask desarrolló **LavaMoat**. Herramientas similares aplican el Principio de Autoridad Mínima de Objetos (Principle of Least Authority - POLA) a nivel del runtime de empaquetado (compatible con Webpack, Browserify o Rollup).

LavaMoat envuelve cada paquete `npm` individual importado en un sandbox restrictivo basado en **Hardened JavaScript (Secure EcmaScript - SES)**. A través de un archivo de configuración de políticas generado automáticamente (`lavamoat-policy.json`), se deniega el acceso a globales del sistema a menos que estén explícitamente autorizadas.

JSON

```
"resources"
  "dependencia-menor-comprometida"
    "globals"
```

```
      "fetch"  false
      "chrome.storage.local"  false
      "document"  false


"core-crypto-wallet-module"
  "globals"
    "crypto.subtle"  true
```

Nota de Seguridad de Arquitectura: Si una dependencia menor maliciosa inyectada en tu entorno de desarrollo intenta leer los inputs de mnemónicos y realizar un `fetch()` hacia un servidor externo para robar las claves, el sandbox de JavaScript Hardened bloqueará inmediatamente la ejecución al detectar que esa librería específica no posee permisos explícitos en la política para interactuar con la red o con el DOM, neutralizando el ataque de forma automatizada y sin intervención del desarrollador.